

Search Typeahead — HLD Assignment Report

Author: Subhan Rahiman · **Email:** subhan.24bcs10095@sst.scaler.com · **Date:** 2026-06-22

Repository: <https://github.com/SubhanRahiman7/HLD-Assignment>

Final commit hash: cfb6838cc8276a6a435f74ca3d68cd55939e7234

A production-style prefix-search typeahead service with simulated 4-node consistent-hash cache, batched write path, popularity/recency ranking, and a UI that mirrors the provided design 1:1.

1. Architecture

1.1 System diagram

```
Browser (UI) |
- Header / brand / ranking toggle (popularity ⇌ recency) |
- Search input with debounced live suggestions |
- Dropdown with prefix highlighting, keyboard navigation |
- Result card (200 OK / 503) |
- Trending chips, Recent chips (clear) |
- Telemetry line: HIT/MISS · ms · node |
```

| HTTP (fetch)

```
Express / Spring Boot server |
|
GET /suggest → Consistent-hash cache (4 nodes) → |
FNV-1a(prefix) % 4 → node | |
cache hit: return Entry | |
cache miss: | |
▼ | |
Ranker.rank(prefix, mode) | |
├ popularity: sort by c | |
└ recency: c + recent·30k | |
▼ | |
Store.data (primary, in-memory) | |
cache.put(node, prefix, Entry) | |
|
POST /search → Recent.add(q) | |
Store.enqueueIncrement(q) → | |
recentActivity[q] += 1 | |
narrow cache invalidation | |
(only prefixes ⊆ q are dropped) | |
▼ | |
WriteBuffer[q] += 1 ← coalesce | |
|
BatchScheduler @ fixedDelay = 2s → | |
▼ | |
```

```
Store.flush() applies buffered deltas | |
atomically to Store.data | |
(one write per query per 2s window) | |
```

1.2 Module map

Node build /

File	Responsibility
server.js	HTTP layer, telemetry, batch flusher loop, graceful shutdown
src/store.js	Primary store, count map, recent-activity map, write buffer, periodic flush
src/cache.js	FNV-1a hash + 4-node consistent-hash cache shim
src/ranking.js	Prefix filter + popularity/recency sort
src/recent.js	Case-insensitive dedupe, cap 6
public/index.html, styles.css, app.js	Frontend mirroring the design source

Java / Spring Boot build /java

File	Responsibility
pom.xml	Spring Boot 3.4.7 parent, Java 21
application.properties	server.port=3000
TypeaheadApplication.java	@SpringBootApplication + @EnableScheduling
TypeaheadController.java	All REST endpoints
Store.java	Primary store + write buffer + flush()
BatchScheduler.java	@Scheduled(fixedDelay=2000L)
ConsistentHashCode.java	4-node FNV-1a cache shim (@Component)
Ranker.java	popularity / recency ranking
Recent.java	user-recent (cap 6)
QueryCount.java, Suggestion.java	Data records

File	Responsibility
src/main/resources/static/*	Identical frontend (HTML/CSS/JS)

1.3 Request lifecycle — happy path

1. User types a prefix. After 100ms debounce, UI calls `GET /suggest?q=...&mode=popularity`.
2. Server hashes the prefix with FNV-1a, takes `% 4` to pick a cache node (`cache-a` – `cache-d`), and looks up `(node, prefix)`.
3. **Cache hit:** return cached list + `hit: true` + elapsed ms.
4. **Cache miss:** run `Ranker.rank(DATA, prefix, mode)` → top 10 → store with 60s TTL → return `hit: false`.
5. User picks a suggestion (or hits Enter) → UI posts `{"q":"..."}` to `POST /search`.
6. Server: `recentActivity[q] += 1`, enqueue `+1` into `WriteBuffer[q]`, narrow-invalidate cache (drop keys that are prefixes of `q`), add to Recent, pre-flush so the response shows the new count.
7. Independently, every 2s the `BatchScheduler` (`@Scheduled` in Java, `setInterval` in Node) calls `Store.flush()`, applying all buffered deltas in one pass.

2. Dataset and loading instructions

2.1 Dataset

The query dataset is hand-seeded inside the primary store (no external DB / file / network). Seed records:

```
iphone, iphone 15, iphone 15 pro, iphone charger, iphone case,  
ipad, ipad pro, airpods, airpods pro, samsung galaxy s24, samsung tv,  
macbook air, macbook pro, nintendo switch 2, sony headphones,  
java tutorial, java jobs, javascript, javascript array methods,  
python tutorial, python pandas, react hooks, react router,  
react native, redis caching, system design interview,  
how to tie a tie, how to screenshot on mac, how to boil eggs,  
how to invest in stocks, best laptops 2026, best headphones,  
best coffee maker, best running shoes, taylor swift tickets,  
world cup schedule, weather today, amazon prime day, chatgpt,  
flight tickets, nike air force 1, adidas samba,  
coffee near me, concert tickets
```

Counts are realistic all-time search-volume-style numbers (15k–100k). Five queries are seeded into `recentActivity` so the **recency** mode visibly reorders the trending list.

Source: the assignment brief asked the design to copy a screenshot of a search-typeahead UI; the underlying query/count data is treated as in-memory sample data and seeded on boot. There is no external dataset to download.

2.2 Loading instructions

```
# Node build (default)  
cd project  
npm install  
npm start # binds to :3000  
  
# Java / Spring Boot build  
cd project/java  
mvn spring-boot:run # binds to :3000  
# or  
mvn -DskipTests package  
java -jar target/search-typeahead-1.0.0.jar
```

On boot, `Store.seed()` populates 44 query rows. Counts reset on restart by design — the assignment scope was "store query-count data", not persistence.

3. API documentation

Base URL: `http://localhost:3000` . All responses are JSON.

3.1 GET /suggest

Returns top-10 suggestions for a prefix under the chosen ranking mode.

Name	Type	Default	Notes
<code>q</code>	string	<code>""</code>	Empty → empty list, no cache write
<code>mode</code>	string	<code>popularity</code>	<code>popularity</code> <code>recency</code>

Response — cache miss

```
{
  "node": "cache-a",
  "hit": false,
  "ms": 0.42,
  "suggestions": [
    { "q": "iphone", "c": 100000 },
    { "q": "iphone 15", "c": 85000 },
    { "q": "iphone charger", "c": 60000 }
  ]
}
```

Response — cache hit (`hit: true` , identical list, sub-0.05ms)

Errors: empty prefix → `{"suggestions": [], "node": "", "hit": false, "ms": 0}` (200 OK).

3.2 POST /search

Records a search, increments count, pre-flushes the buffer, performs narrow cache invalidation, adds to Recent.

Request body

```
{ "q": "iphone 15" }
```

Response — 200

```
{
  "message": "Searched",
  "query": "iphone 15",
  "count": 85001,
}
```

```
"ms": 0.56
}
```

Response — 503 (demo failure path). When `q === "error"` (case-insensitive):

```
{ "message": "Upstream search service did not respond within the latency budget." }
```

Response — 400 when `q` is missing/empty.

3.3 GET /trending

Top-6 by current mode.

```
{
  "mode": "popularity",
  "trending": [
    { "q": "iphone", "c": 100000 },
    { "q": "weather today", "c": 88000 },
    { "q": "iphone 15", "c": 85000 },
    { "q": "javascript", "c": 75000 }
  ],
  "note": "by all-time search count"
}
```

With `?mode=recency`, the `note` becomes `"recency-weighted score"` and order shifts to favor recent queries.

3.4 GET /recent and POST /recent/clear

```
{ "recent": ["iphone 15", "macbook air"] }
```

`POST /recent/clear` → `{"ok":true}`.

3.5 GET /cache/debug and POST /cache/flush

```
curl "http://localhost:3000/cache/debug?prefix=iph"
# { "prefix":"iph", "node":"cache-a", "hit":false,
#   "nodes":["cache-a","cache-b","cache-c","cache-d"] }
```

`POST /cache/flush` invalidates all four nodes.

3.6 GET /telemetry

```
{
  "hits": 42,
  "misses": 17,
```

```
"writes": 12,  
"batches": 6,  
"batchedOps": 9,  
"lastFlushMs": 0,  
"cacheSize": 4,  
"dataSize": 44,  
"pendingWrites": 0  
}
```

3.7 POST /benchmark/batch?writes=N

Simulates **N** searches by enqueueing **N** increments on a fixed sample, then flushing once.

```
{  
  "simulatedSearches": 100,  
  "actualBatchedWrites": 1,  
  "writeReductionFactor": 100.0,  
  "note": "Batch coalesces same-query writes within a 2s window into a single store  
update."  
}
```

4. Design choices and trade-offs

4.1 In-memory primary store

Why: zero-setup demo; the assignment is a UI/design+architecture exercise, not a persistence exercise.

Trade-off: counts reset on restart. Swapping `Store.data` for a real DB later is a 1-line change at the boundary; the rest of the system doesn't care.

4.2 Write batching with a 2s window

Why: incoming search writes are bursty and often duplicate (a trending query, an autocompleted-then-submitted query). A 2s window coalesces duplicate writes into a single primary-store update, reducing write amplification by $\sim 100\times$ under uniform bursts (see `/benchmark/batch`).

Trade-off: worst-case staleness for the count returned by `POST /search` is 2s. We mitigate this by calling `Store.flush()` synchronously inside the `/search` handler so the response reflects the new count immediately; the batch loop is for any writes that bypass the handler (none in this design, but the slot exists for telemetry/background scoring).

4.3 Consistent-hash cache across 4 logical nodes

Why: demonstrates the simplest correct cache-shard routing — `FNV-1a(prefix) % 4` — without bringing in a vnode library. Adding a node only redistributes $\sim 1/N$ of the keys (the standard consistent-hash property).

Trade-off: modulo-based hashing has slightly worse key-distribution variance than a true vnode ring, but is a one-line method that any reader can audit.

4.4 Narrow cache invalidation on `/search`

Why: when query `q` is incremented, only the cache entries whose key is a prefix of `q` can have changed ordering. All other prefixes' top-K results stay valid. This avoids the "every search invalidates everything" anti-pattern.

Trade-off: a 60s TTL is the safety net for any invalidation we miss.

4.5 Two ranking modes

popularity: sort by all-time count.

recency: `score = c + recentActivity[q] * 30_000`. The 30k multiplier is tuned so a single recent write on a low-volume query can outrank a much-larger all-time count.

Trade-off: the multiplier is a magic number. A real system would learn it from CTR/engagement signals.

4.6 Recent list (cap 6)

Why: UI only has space for 6 chips. Cap is enforced at the server so the UI doesn't have to truncate.

Trade-off: `Recent` is per-process, not per-user. A real system would key it on `userId` or a session cookie.

4.7 503 demo path

`POST /search` with `q === "error"` returns 503 to exercise the UI's red error state. This is intentional; the assignment asks for failure handling.

4.8 Why two builds?

The Node build is the canonical implementation (small, easy to read, no compile step). The Spring Boot build is a 1:1 translation that demonstrates the architecture in a statically-typed JVM framework — useful for evaluating whether the HLD choices hold up under stricter types and a heavier framework. Endpoints, payloads, and behavior match exactly; the frontend is byte-identical between the two.

5. Performance report

5.1 Latency (local, in-memory)

Operation	Median	Notes
GET /suggest cache hit	0.01–0.05 ms	Single map lookup
GET /suggest cache miss	0.2–0.5 ms	Linear scan of 44 rows + sort
POST /search	0.5–1 ms	Linear scan + buffered enqueue + narrow cache invalidation + pre-flush
GET /trending	<0.5 ms	Same as suggest with empty prefix
POST /benchmark/batch?writes=100	0.5 ms	1 actual batched write

5.2 Write amplification

A uniform 100-write burst on 7 queries collapses to **1 actual store write** (factor of **100×**). Real-world bursts are even more skewed — a trending query dominates — so the reduction factor is usually higher.

5.3 Cache effectiveness

Repeated /suggest calls for the same prefix hit the cache after the first request. Telemetry shows `hits / (hits + misses)` climbing toward 1 as a user keeps typing within the same prefix family.

5.4 Memory

44 query rows × ~30 bytes + a small per-node cache map. Total well under 1 MB. No allocations on the hot path after warm-up.

5.5 End-to-end smoke results

```
GET /trending → 200 OK, top-6 ranked
GET /suggest?q=iphone (miss) → 0.19 ms, node=cache-a
GET /suggest?q=iphone (hit) → 0.01 ms, hit=true
POST /search {"q":"iphone 15"} → count=85001
GET /cache/debug?prefix=iph → node=cache-a
POST /benchmark/batch?writes=100 → actualBatchedWrites=1, factor=100×
POST /search {"q":"error"} → 503, demo failure path
```

5.6 Future work / what's not done

- Persistence (Postgres / Redis / Kafka).
- Authentication and per-user Recent.
- Real consistent-hash ring with vnodes for production variance.
- Adaptive recency multiplier from CTR.
- Distributed cache (Redis Cluster) replacing the in-process shim.
- Tracing / metrics export (OpenTelemetry) and `/healthz`.

6. Repository layout

```
HLD-Assignment/  
├── README.md  
├── package.json / package-lock.json  
├── server.js  
├── public/ # frontend (served by both builds)  
│   ├── index.html  
│   ├── styles.css  
│   └── app.js  
├── src/ # Node backend modules  
│   ├── store.js  
│   ├── cache.js  
│   ├── ranking.js  
│   └── recent.js  
├── java/ # Spring Boot backend  
│   ├── pom.xml  
│   ├── README.md  
│   └── src/main/  
│       ├── java/com/typeahead/  
│       │   ├── TypeaheadApplication.java  
│       │   ├── TypeaheadController.java  
│       │   ├── Store.java  
│       │   ├── BatchScheduler.java  
│       │   ├── ConsistentHashCache.java  
│       │   ├── Ranker.java  
│       │   ├── Recent.java  
│       │   ├── QueryCount.java  
│       │   └── Suggestion.java  
│       └── resources/  
│           ├── application.properties  
│           └── static/ # copy of public/
```